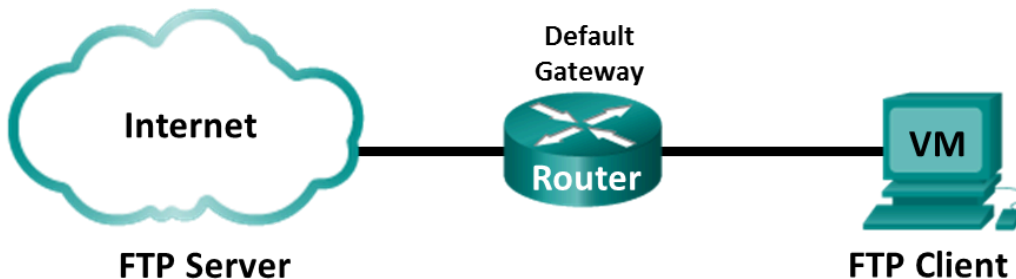


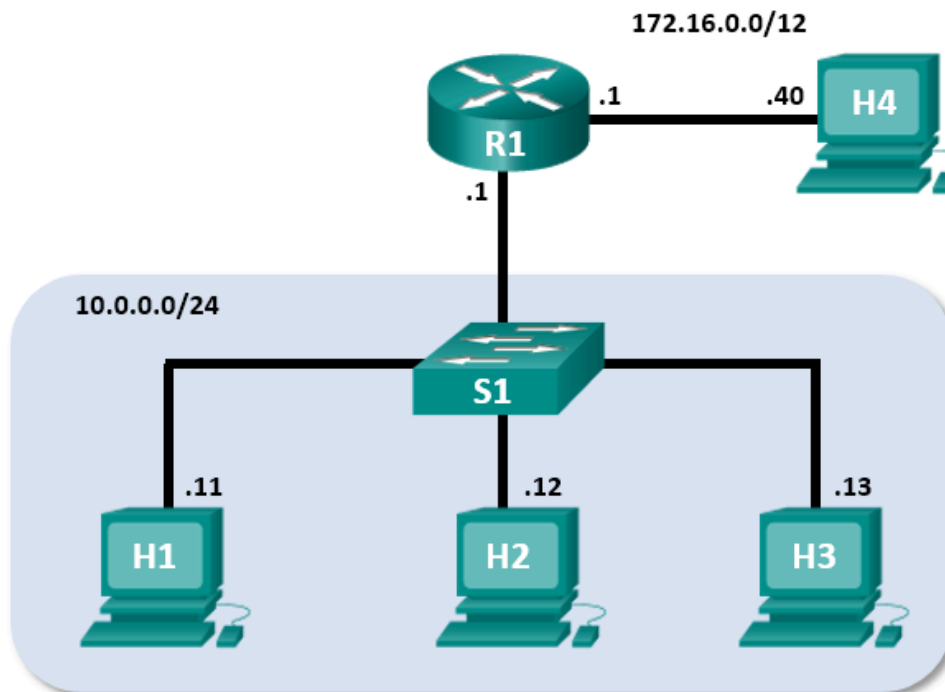
Використання Wireshark для дослідження TCP та UDP захоплених пакетів

Топологія - Частина 1 (FTP)



Частина 1 присвячена TCP-захопленню сеансу FTP. Ця топологія складається з віртуальної машини CyberOps Workstation з доступом в Інтернет.

Топологія Mininet - частина 2 (TFTP)



Цілі та задачі

Частина 1: Визначити поля TCP заголовку та операції, використовуючи Wireshark захоплення FTP сеансу

Частина 2: Визначити поля UDP заголовку та операції, використовуючи Wireshark захоплення TFTP сеансу

Довідкова інформація / Сценарій

Двома протоколами транспортного рівня TCP/IP є TCP (визначено в RFC 761) і UDP (визначено в RFC 768). Обидва протоколи підтримують обмін даними протоколом верхнього рівня. Наприклад, TCP використовується для забезпечення підтримки транспортного рівня для протоколів передачі гіпертексту (HTTP) і FTP, серед інших. UDP забезпечує підтримку транспортного рівня для системи доменних імен (DNS) і TFTP, серед інших.

У частині 1 цієї лабораторії ви будете використовувати інструмент Wireshark з відкритим вихідним кодом для захоплення та аналізу полів заголовків протоколу TCP для передачі файлів FTP між головним комп'ютером і анонімним FTP-сервером. Командний рядок терміналу використовується для під'єднання до анонімного FTP-сервера і завантаження файлу. У частині 2 цієї лабораторії ви будете використовувати Wireshark для захоплення та аналізу полів заголовка UDP для передачі файлів TFTP між двома хост-комп'ютерами Mininet.

Необхідні ресурси

- CyberOps Workstation VM
- Доступ до Інтернету

Інструкції

Частина 1: Визначити поля TCP заголовку та операції, використовуючи Wireshark захоплення FTP сеансу

У частині 1 використовується Wireshark для захоплення сеансу FTP та перевірки полів заголовка TCP.

Крок 1: Почніть перехоплення даних програмою Wireshark.

- Запустіть і увійдіть у віртуальну машину CyberOps Workstation. Відкрийте вікно терміналу і запустіть програму Wireshark. Амперсанд (&) відправляє процес у фоновий режим і дозволяє продовжувати роботу в тому самому терміналі.

```
[analyst@secOps ~]$ wireshark &
```

- Запустіть перехоплення даних програмою Wireshark для інтерфейсу **enp0s3**.

- Відкрийте інше вікно терміналу для доступу до зовнішнього сайту ftp. Введіть **ftp ftp.cdc.gov** в рядку запиту. Увійдіть на FTP-сайт Центру з контролю та профілактики захворювань (Centers for Disease Control and Prevention, CDC) як користувач **anonymous** та без пароля.

```
[analyst@secOps ~]$ ftp ftp.cdc.gov
Connected to ftp.cdc.gov.
220 Microsoft FTP Service
Name (ftp.cdc.gov:analyst): anonymous
331 Anonymous access allowed, send identity (e-mail name) as password.
Password:
230 User logged in.
Remote system type is Windows_NT.
```

```
ftp>
```

Крок 2: Завантажте файл Readme.

- a. Знайдіть і завантажте файл Readme, ввівши команду **ls**, щоб переглянути список файлів.

```
ftp> ls
200 PORT command successful.
125 Data connection already open; Transfer starting.
-rwxrwxrwx 1 owner group 128 May 9 1995 .change.dir
-rwxrwxrwx 1 owner group 107 May 9 1995 .message
drwxrwxrwx 1 owner group 0 Feb 2 11:21 pub
-rwxrwxrwx 1 owner group 1428 May 13 1999 Readme
-rwxrwxrwx 1 owner group 383 May 13 1999 Siteinfo
-rwxrwxrwx 1 owner group 0 May 17 2005 up.htm
drwxrwxrwx 1 owner group 0 May 20 2010 w3c
-rwxrwxrwx 1 owner group 202 Sep 22 1998 welcome.msg
226 Transfer complete.
```

Примітка: Може з'явитися такі повідомлення:

```
421 Service not available, remote server has closed connection
ftp: No control connection for command
```

```
501 Server cannot access argument
500 command not understood
ftp: bind: Address already in use
```

Якщо це станеться, FTP-сервер наразі не працює. Однак ви можете продовжити виконувати решту лабораторної роботи, аналізуючи ті пакети, які ви змогли захопити, і зчитуючи пакети, які ви не захопили. Ви також можете повернутися до виконання лабораторної роботи пізніше, щоб перевірити, чи відновлено FTP-сервер.

- b. Введіть команду **get Readme** для завантаження файлу. Коли завантаження завершиться, введіть команду **quit**, щоб вийти. (Примітка: Якщо ви не можете завантажити файл, ви можете продовжити роботу з іншою частиною лабораторної роботи.)

```
ftp> get Readme
200 PORT command successful.
125 Data connection already open; Transfer starting.
WARNING! 36 bare linefeeds received in ASCII mode
File may not have transferred correctly.
226 Transfer complete.
1428 bytes received in 0.056 seconds (24.9 kbytes/s)
```

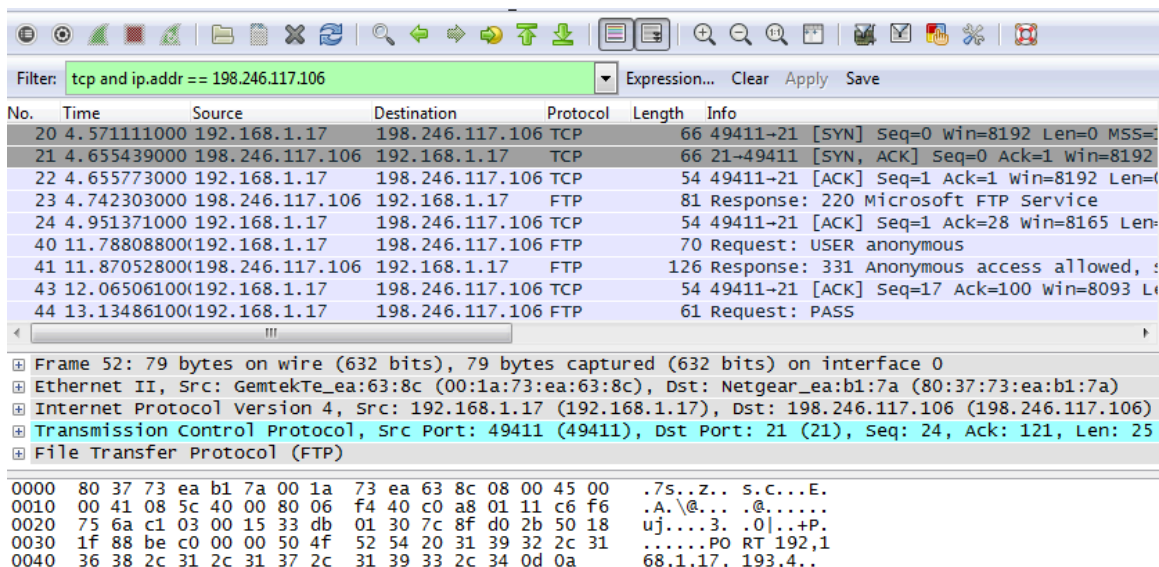
- c. Після завершення передачі для виходу з ftp введіть **quit**.

Крок 3: Зупиніть перехоплення даних програмою Wireshark.

Крок 4: Перегляд головного вікна Wireshark.

Wireshark захопив багато пакетів під час сеансу FTP на ftp.cdc.gov. Щоб обмежити обсяг даних для аналізу, застосуйте фільтр **tcp and ip.addr == 198.246.117.106** і натисніть **Apply (Застосувати)**.

Примітка: IP-адреса, 198.246.117.106, є адресою ftp.cdc.gov на момент створення цієї лабораторної роботи. У вас IP-адреса може відрізнятися. Якщо так, знайдіть перший пакет TCP, який почав тристороннє рукоштовування з ftp.cdc.gov. IP-адреса призначення — це IP-адреса, яку слід використовувати для вашого фільтра.



Примітка.: Інтерфейс Wireshark може виглядати трохи інакше, ніж наведене вище зображення.

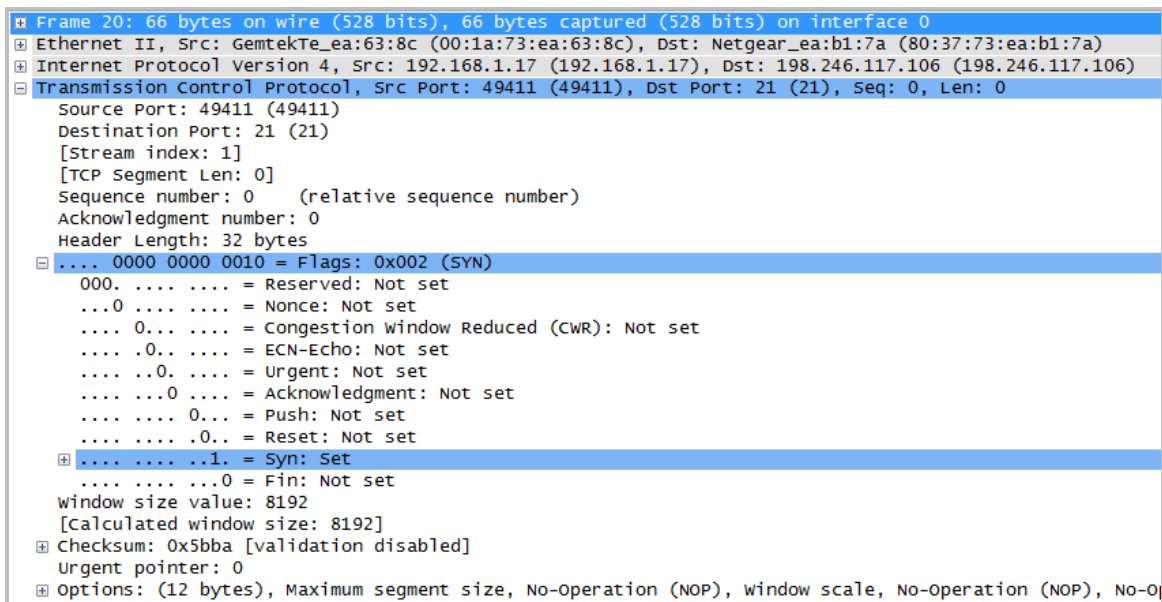
Крок 5: Проаналізуйте поля TCP.

Після того, як фільтр TCP застосовано, перші три пакети (верхня частина) відображають послідовність [SYN], [SYN, ACK] і [ACK], яка є тристороннім рукоштовуванням TCP.

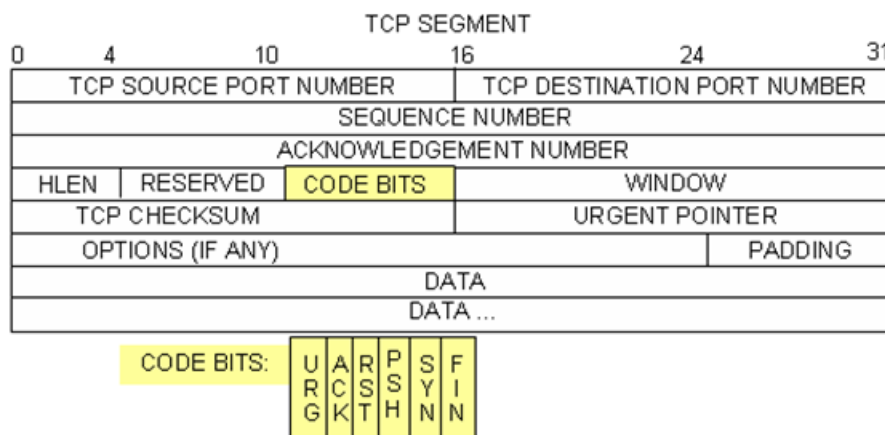
20	4.571111000	192.168.1.17	198.246.117.106	TCP	66	49411-21 [SYN] Seq=0 win=8192 Len=0 MSS=
21	4.655439000	198.246.117.106	192.168.1.17	TCP	66	21-49411 [SYN, ACK] Seq=0 Ack=1 win=8192
22	4.655773000	192.168.1.17	198.246.117.106	TCP	54	49411-21 [ACK] Seq=1 Ack=1 win=8192 Len=

TCP зазвичай використовується під час сеансу для керування доставкою даних, перевірки надходження даних і керування розміром вікна. Для кожного обміну даними між FTP-клієнтом і FTP-сервером запускається новий сеанс TCP. Після завершення передачі даних сеанс TCP закривається. Коли сеанс FTP завершується, TCP виконує впорядковане завершення роботи та припинення.

У Wireshark докладна інформація про TCP доступна в області відомостей про пакет (середня частина). Виділіть першу датаграму TCP з головного комп'ютера та розгорніть її частини, як показано нижче.



Розгорнута датаграма TCP виглядає подібно до панелі деталей пакета, як показано нижче.



Зображення вище є діаграмою датаграми TCP. Пояснення кожного поля наведено для довідки:

- **Номер порту джерела TCP (TCP source port number)** належить хосту сеансу TCP, який відкрив з'єднання. Зазвичай це випадкове значення вище 1023.
- **Номер порту призначення TCP (TCP destination port number)** використовується для ідентифікації верхнього рівня протоколу або застосунку на віддаленому сайті. Значення в діапазоні 0–1023 представляють «добре відомі порти» та пов'язані з популярними сервісами та застосунками (як описано у RFC 1700), такими як Telnet, FTP і HTTP. Поеднання IP-адреси джерела, порту джерела, IP-адреси призначення та порту призначення унікально ідентифікує сеанс для відправника та одержувача.

Примітка: У наведеному вище записі Wireshark порт призначення - 21, тобто це FTP. FTP-сервери прослуховують порт 21 для FTP-клієнтських з'єднань.

- **Номер послідовності (Sequence number)** визначає номер останнього октету в сегменті.
- **Номер підтвердження (Acknowledgment number)** визначає наступний октет, очікуваний одержувачем.
- **Біти коду (Code bits)** мають особливе значення в керуванні сеансами і в обробці сегментів. Серед цікавих значень можна виділити:
 - **ACK** — підтвердження отримання сегмента.
 - **SYN** — синхронізація, встановлюється лише тоді, коли новий сеанс TCP узгоджується під час TCP тристороннього рукоятискання.
 - **FIN** — Закінчення, запит на закриття сеансу TCP.
- **Розмір вікна (Window size)** - це значення розсувного вікна. Воно визначає, скільки октетів може бути надіслано перед очікуванням підтвердження.
- **Показник Термінове (Urgent)** використовується лише з позначкою «Термінове» (URG), коли відправнику потрібно надіслати термінові дані одержувачу.
- **Параметри (Options)** наразі мають лише один параметр, і він визначається як максимальний розмір сегмента TCP (необов'язкове значення).

Використовуючи захоплення Wireshark першого запуску сеансу TCP (SYN біт встановлено 1), заповніть інформацію про заголовок TCP. Деякі поля можуть не застосовуватися у цьому пакеті.

Від VM до сервера CDC (тільки SYN біт встановлений в 1):

Опис	Результати Wireshark (приклад)
IP-адреса джерела	IP-адреса вашої VM
IP-адреса призначення	198.246.117.106 (IP-адреса ftp.cdc.gov)
Номер порту джерела	Випадковий порт > 1023 (наприклад, 49344)
Номер порту призначення	21 (стандартний порт FTP)
Порядковий номер	0
Номер підтвердження	0
Довжина заголовка	20 байтів (або більше, якщо є опції)
Розмір вікна	~65535 (або інше велике значення)

Экспортировать в Таблицы

У другому відфільтрованому захопленні Wireshark FTP-сервер CDC підтверджує запит від віртуальної машини. Зверніть увагу на значення бітів SYN і ACK.

```

+ Frame 21: 66 bytes on wire (528 bits), 66 bytes captured (528 bits) on interface 0
+ Ethernet II, Src: Netgear_ea:b1:7a (80:37:73:ea:b1:7a), Dst: GemtekTe_ea:63:8c (00:1a:73:ea:63:8c)
+ Internet Protocol Version 4, Src: 198.246.117.106 (198.246.117.106), Dst: 192.168.1.17 (192.168.1.17)
+ Transmission Control Protocol, Src Port: 21 (21), Dst Port: 49411 (49411), Seq: 0, Ack: 1, Len: 0
  Source Port: 21 (21)
  Destination Port: 49411 (49411)
  [Stream index: 1]
  [TCP segment Len: 0]
  Sequence number: 0 (relative sequence number)
  Acknowledgment number: 1 (relative ack number)
  Header Length: 32 bytes
  .... 0000 0001 0010 = Flags: 0x012 (SYN, ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 = Acknowledgment: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    + .... .... ..1. = Syn: Set
    .... .... ...0 = Fin: Not set
  window size value: 8192
  [Calculated window size: 8192]
  + Checksum: 0x0ee7 [validation disabled]
  Urgent pointer: 0
  + Options: (12 bytes), Maximum segment size, No-operation (NOP), window scale, No-operation (NOP), No
  + [SEQ/ACK analysis]

```

Заповніть наступну таблицю інформацією про повідомлення SYN-ACK.

Опис	Результати Wireshark (приклад)
IP-адреса джерела	198.246.117.106 (IP-адреса ftp.cdc.gov)
IP-адреса призначення	IP-адреса вашої VM
Номер порту джерела	21
Номер порту призначення	Той самий порт джерела з пакета 1
Порядковий номер	0
Номер підтвердження	1
Довжина заголовка	20 байтів (або більше)
Розмір вікна	~28960 (або інше значення, встановлене сервером)

На заключному етапі узгодження для встановлення зв'язку віртуальна машина надсилає на сервер повідомлення з підтвердженням. Зверніть увагу, що тільки біт ACK біт має значення 1, і порядковий номер було збільшено до 1.

```

+ Frame 22: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0
+ Ethernet II, Src: GemtekTe_ea:63:8c (00:1a:73:ea:63:8c), Dst: Netgear_ea:b1:7a (80:37:73:ea:b1:7a)
+ Internet Protocol Version 4, Src: 192.168.1.17 (192.168.1.17), Dst: 198.246.117.106 (198.246.117.106)
- Transmission Control Protocol, Src Port: 49411 (49411), Dst Port: 21 (21), Seq: 1, Ack: 1, Len: 0
  Source Port: 49411 (49411)
  Destination Port: 21 (21)
  [Stream index: 1]
  [TCP Segment Len: 0]
  Sequence number: 1 (relative sequence number)
  Acknowledgment number: 1 (relative ack number)
  Header Length: 20 bytes
  - ... 0000 0001 0000 = Flags: 0x010 (ACK)
    000. .... = Reserved: Not set
    ...0 .... = Nonce: Not set
    .... 0... = Congestion window Reduced (CWR): Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...1 .... = Acknowledgment: Set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    .... .... ..0. = Syn: Not set
    .... .... ...0 = Fin: Not set
  window size value: 8192
  [calculated window size: 8192]
  [window size scaling factor: 1]
  + Checksum: 0x4f6a [validation disabled]
  urgent pointer: 0
  + [SEQ/ACK analysis]

```

Заповніть наведену нижче таблицю інформацією про повідомлення ACK.

Опис	Результати Wireshark (приклад)
IP-адреса джерела	IP-адреса вашої VM
IP-адреса призначення	198.246.117.106 (IP-адреса ftp.cdc.gov)
Номер порту джерела	Той самий, що й у пакеті 1
Номер порту призначення	21
Порядковий номер	1
Номер підтвердження	1
Довжина заголовка	20 байтів

Розмір вікна	~237 (або інше значення, встановлене VM)
--------------	--

Скільки інших дейтаграм TCP містили біт SYN?

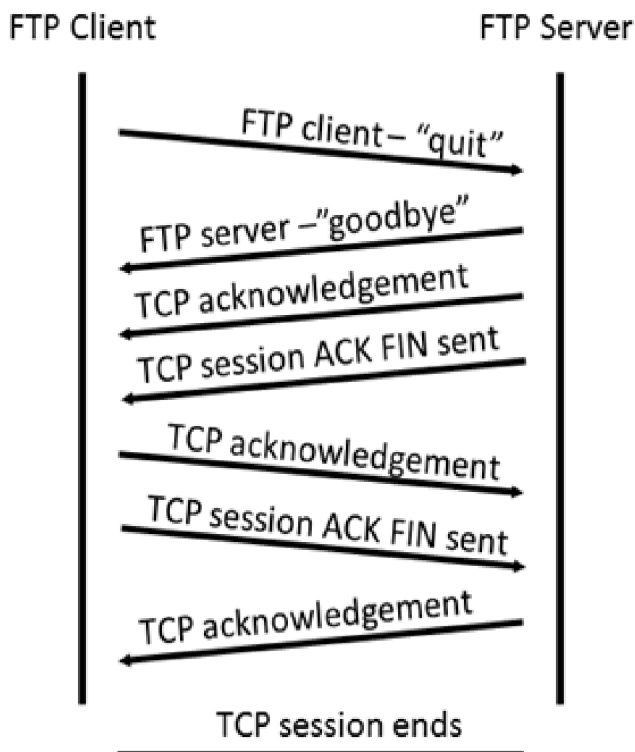
Жодної. Біт SYN використовується тільки на початку для встановлення нового з'єднання. Після завершення тристороннього рукоштовування він більше не встановлюється для цього сеансу.

Після встановлення TCP-сеансу між ПК і FTP-сервером може передаватися FTP-трафік. FTP-клієнт і сервер спілкуються один з одним, не знаючи, що TCP має контроль і керування сеансом. Коли сервер FTP надсилає *відповідь: 220* клієнту FTP, сеанс TCP на клієнті FTP надсилає підтвердження сеансу TCP на сервері. Ця послідовність відображається в захопленні Wireshark нижче.

23	4.742303000	198.246.117.106	192.168.1.17	FTP	81	Response: 220 Microsoft FTP Service
24	4.951371000	192.168.1.17	198.246.117.106	TCP	54	49411-21 [ACK] Seq=1 Ack=28 win=8165 Len=
40	11.788088000	192.168.1.17	198.246.117.106	FTP	70	Request: USER anonymous
41	11.870528000	198.246.117.106	192.168.1.17	FTP	126	Response: 331 Anonymous access allowed, :

Frame 23: 81 bytes on wire (648 bits), 81 bytes captured (648 bits) on interface 0 Ethernet II, Src: Netgear_ea:b1:7a (80:37:73:ea:b1:7a), Dst: GemtekTe_ea:63:8c (00:1a:73:ea:63:8c) Internet Protocol version 4, Src: 198.246.117.106 (198.246.117.106), Dst: 192.168.1.17 (192.168.1.17) Transmission Control Protocol, Src Port: 21 (21), Dst Port: 49411 (49411), Seq: 1, Ack: 1, Len: 27 File Transfer Protocol (FTP) 220 Microsoft FTP Service\r\n Response code: Service ready for new user (220) Response arg: Microsoft FTP Service	
--	--

Після завершення сеансу FTP клієнт FTP надсилає команду «вийти». Сервер FTP підтверджує завершення роботи FTP *відповіддю: 221 Goodbye*. У цей час TCP-сеанс FTP-сервера надсилає TCP-дейтаграму FTP-клієнту, сповіщаючи про завершення TCP-сеансу. TCP-сеанс FTP-клієнта підтверджує отримання дейтаграми завершення, а потім надсилає власне завершення TCP-сеансу. Коли ініціатор завершення TCP (сервер FTP) отримує дублікат завершення, надсилається дейтаграма ACK для підтвердження завершення, і сеанс TCP закривається. Цю послідовність видно на схемі та зображенні нижче.



Застосувавши фільтр **ftp**, у Wireshark можна перевірити всю послідовність трафіку FTP. Зверніть увагу на послідовність подій під час цього сеансу FTP. Для отримання файлу Readme було використано ім'я користувача **anonymous**. Після завершення передачі файлів користувач завершив сеанс FTP.

No.	Time	Source	Destination	Protocol	Length	Info
23	4.742303000	198.246.117.106	192.168.1.17	FTP	81	Response: 220 Microsoft FTP Service
40	11.788088000	198.246.117.106	192.168.1.17	FTP	70	Request: USER anonymous
41	11.870528000	198.246.117.106	192.168.1.17	FTP	126	Response: 331 Anonymous access allowed, send
44	13.134861000	198.246.117.106	192.168.1.17	FTP	61	Request: PASS
46	13.328294000	198.246.117.106	192.168.1.17	FTP	75	Response: 230 User logged in.
51	16.352248000	198.246.117.106	192.168.1.17	FTP	79	Request: PORT 192,168,1,17,193,4
52	16.682680000	198.246.117.106	192.168.1.17	FTP	79	[TCP Retransmission] Request: PORT 192,168,1,17,193,5
54	17.354538000	198.246.117.106	192.168.1.17	FTP	84	[TCP Retransmission] Response: 200 PORT complete
55	17.363442000	198.246.117.106	192.168.1.17	FTP	60	Request: NLST
56	17.442635000	198.246.117.106	192.168.1.17	FTP	95	Response: 150 opening ASCII mode data connection
62	19.897441000	198.246.117.106	192.168.1.17	FTP	78	Response: 226 Transfer complete.
73	24.297181000	198.246.117.106	192.168.1.17	FTP	79	Request: PORT 192,168,1,17,193,5
75	24.607498000	198.246.117.106	192.168.1.17	FTP	79	[TCP Retransmission] Request: PORT 192,168,1,17,193,5
82	25.136886000	198.246.117.106	192.168.1.17	FTP	84	[TCP Retransmission] Response: 200 PORT complete
83	25.142329000	198.246.117.106	192.168.1.17	FTP	67	Request: RETR Readme
101	25.270185000	198.246.117.106	192.168.1.17	FTP	95	Response: 150 opening ASCII mode data connection
127	27.784523000	198.246.117.106	192.168.1.17	FTP	78	Response: 226 Transfer complete.
147	30.482992000	198.246.117.106	192.168.1.17	FTP	60	Request: QUIT
148	30.565117000	198.246.117.106	192.168.1.17	FTP	68	Response: 221 Goodbye.

Знову застосуйте фільтр TCP у Wireshark, щоб перевірити завершення сеансу TCP. Для завершення сеансу TCP передається чотири пакети. Оскільки з'єднання TCP є повнодуплексним, кожен напрямок має завершуватися незалежно. Перевірте адреси джерела та призначення.

У цьому прикладі FTP-сервер не має більше даних для надсилання в потоці. Він надсилає сегмент із встановленим прапором FIN у кадрі 149. Для підтвердження отримання FIN для завершення сеансу від сервера до клієнта ПК надсилає ACK у кадрі 150.

У кадрі 151 ПК надсилає FIN на FTP-сервер, щоб завершити сеанс TCP. FTP-сервер у кадрі 152 у відповідь надсилає ACK, щоб підтвердити FIN від ПК. Тепер TCP-сесія припиняється між FTP-сервером і ПК.

147	30.48299200	(192.168.1.17	198.246.117.106	FTP	60	Request: QUIT
148	30.56511700	(198.246.117.106	192.168.1.17	FTP	68	Response: 221 Goodbye.
149	30.56646700	(198.246.117.106	192.168.1.17	TCP	54	21→49411 [FIN, ACK] Seq=325 Ack=99 win=1
150	30.56653200	(192.168.1.17	198.246.117.106	TCP	54	49411→21 [ACK] Seq=99 Ack=326 win=7868 Len=0
151	30.56679900	(192.168.1.17	198.246.117.106	TCP	54	49411→21 [FIN, ACK] Seq=99 Ack=326 win=7868 Len=0
152	30.66777000	(198.246.117.106	192.168.1.17	TCP	54	21→49411 [ACK] Seq=326 Ack=100 win=132096 Len=0

Frame 149: 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface 0
 Ethernet II, Src: Netgear_ea:b1:7a (80:37:73:ea:b1:7a), Dst: GemtekTe_ea:63:8c (00:1a:73:ea:63:8c)
 Internet Protocol Version 4, Src: 198.246.117.106 (198.246.117.106), Dst: 192.168.1.17 (192.168.1.17)
 Transmission Control Protocol, Src Port: 21 (21), Dst Port: 49411 (49411), Seq: 325, Ack: 99, Len: 0

Частина 2: Визначити поля UDP заголовку та операції, використовуючи Wireshark захоплення TFTP сеансу

У частині 2 використовується Wireshark для захоплення сеансу TFTP і перевірки полів заголовка UDP.

Крок 1: Запуск сервісів Mininet і tftpd.

- a. Запустіть Mininet. Введіть пароль **cyberops**, коли з'явиться підказка.

```
[analyst@secOps ~]$ sudo lab.support.files/scripts/cyberops_topo.py
[sudo] password for analyst:
```

- b. Запустіть H1 і H2 у рядку **mininet>**.

```
*** Starting CLI:
mininet> xterm H1 H2
```

- c. У вікні терміналу **H1** запустіть сервер tftpd, використовуючи наданий скрипт.

```
[root@secOps analyst]# /home/analyst/lab.support.files/scripts/start_tftpd.sh
[root@secOps analyst]#
```

Крок 2: Створення файлу для передачі tftp.

- a. Створіть текстовий файл у рядку терміналу **H1** у каталозі **/srv/tftp/**.

```
[root@secOps analyst]# echo "This file contains my tftp data." >
/srv/tftp/my_tftp_data
```

- b. Переконайтеся, що файл був створений з потрібними даними в каталозі.

```
[root@secOps analyst]# cat /srv/tftp/my_tftp_data
Цей файл містить мої tftp-дані.
```

- c. Через заходи безпеки для цього конкретного tftp-сервера ім'я приймаючого файлу має вже існувати. На **H2** створіть файл з ім'ям **my_tftp_data**.

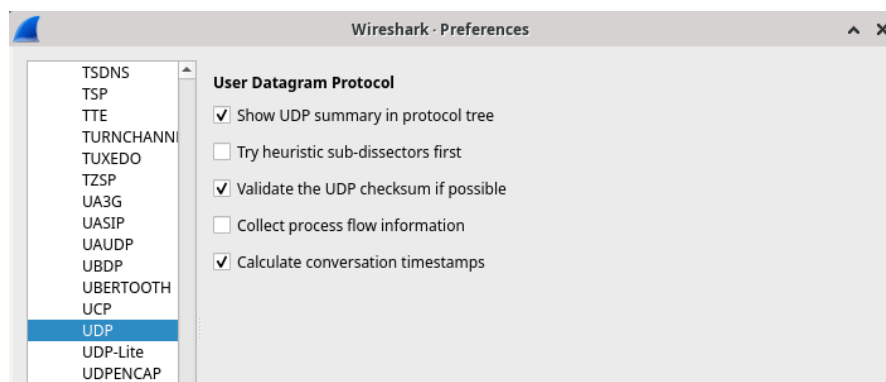
```
[root@secOps analyst]# touch my_tftp_data
```

Крок 3: Захоплення сеансу TFTP у Wireshark

- a. Запустіть Wireshark в H1.

```
[root@secOps analyst]# wireshark &
```

- b. У меню **Редагування (Edit)** виберіть пункт **Параметри (Preferences)** та клацніть стрілку, щоб розгорнути **Протоколи (Protocols)**. Прокрутіть вниз і виберіть **UDP**. Установіть прапорець **Перевіряти контрольну суму UDP, якщо можливо (Validate the UDP checksum if possible)**, і натисніть кнопку **ОК**.



- c. Запустіть перехоплення даних програмою Wireshark для інтерфейсу **H1-eth0**.
 d. Запустіть tftp-сеанс від **H2** до tftp-сервера на **H1** і отримайте файл **my_tftp_data**.

```
[root@secOps analyst]# tftp 10.0.0.11 -c get my_tftp_data
```

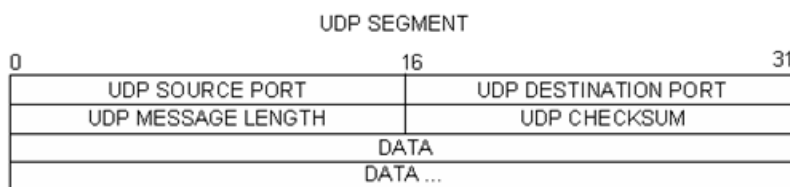
- e. Зупиніть перехоплення даних програмою Wireshark. Встановіть фільтр на **tftp** і натисніть **Застосувати (Apply)**. Скористайтеся трьома пакетами TFTP, щоб заповнити таблицю та відповісти на запитання в решті частини цієї лабораторної роботи.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.0.0.12	10.0.0.11	TFTP	66	Read Request, File: my_tftp_data, Transfer
2	0.001295043	10.0.0.11	10.0.0.12	TFTP	80	Data Packet, Block: 1 (last)
3	0.001735272	10.0.0.12	10.0.0.11	TFTP	46	Acknowledgement, Block: 1

Детальна інформація про UDP доступна на панелі деталей про пакети Wireshark. Виділіть першу дейтаграму UDP з головного комп'ютера та перемістіть вказівник миші на панель деталей пакета. Можливо, знадобиться налаштувати панель деталей пакета та розгорнути запис UDP, натиснувши на поле розгортання протоколу. Розгорнута дейтаграма UDP має виглядати подібно до наведеної нижче схеми.

UDP Header	▼ User Datagram Protocol, Src Port: 47844, Dst Port: 69 Source Port: 47844 Destination Port: 69 Length: 32 ▶ Checksum: 0x2029 [correct] [Checksum Status: Good] [Stream index: 0]
UDP Data	▼ Trivial File Transfer Protocol Opcode: Read Request (1) Source File: my_tftp_data Type: netascii

На рисунку нижче представлена схема дейтаграми UDP. Інформація заголовка мізерна порівняно з датаграмою TCP. Подібно до TCP, кожна датаграма UDP ідентифікується портом джерела UDP і портом призначення UDP.



Використовуючи захоплення Wireshark першої дейтаграми UDP, заповніть інформацію про заголовок UDP. Значення контрольної суми - це шістнадцяткове (базове 16) значення, позначене кодом 0x перед значенням:

Опис	Результати Wireshark (приклад)
IP-адреса джерела	10.0.0.12 (H2)
IP-адреса призначення	10.0.0.11 (H1)
Номер порту джерела	Випадковий динамічний порт (напр., 45193)
Номер порту призначення	69 (стандартний порт TFTP)

Довжина повідомлення UDP	36 байтів
Контрольна сума UDP	0x3d32 (або інше шістнадцяткове значення)

Запитання:

Як UDP перевіряє цілісність дейтаграми?

UDP використовує поле **"Контрольна сума" (Checksum)** у своєму заголовку. Відправник обчислює контрольну суму на основі вмісту заголовка та даних, а одержувач виконує те ж обчислення. Якщо результати не збігаються, дейтаграма вважається пошкодженою.

Перевірте перший кадр, який повертається з сервера tftpd. Заповніть інформацію про заголовок UDP:

Опис	Результати Wireshark (приклад)
IP-адреса джерела	10.0.0.11 (H1)
IP-адреса призначення	10.0.0.12 (H2)
Номер порту джерела	Випадковий динамічний порт (напр., 38555)
Номер порту призначення	Той самий порт джерела з пакета 1 (напр., 45193)
Довжина повідомлення UDP	544 байти
Контрольна сума UDP	0x82b2 [неправильно]

Зверніть увагу, що у дейтаграми UDP, що повертається, інший порт джерела UDP, але цей порт джерела використовується для решти TFTP передачі. Оскільки немає надійного з'єднання, для підтримки передачі TFTP використовується лише оригінальний порт джерела, який використовується для початку сеансу TFTP.

Крім того, зверніть увагу, що контрольна сума UDP неправильна. Це, швидше за все, викликано розвантаженням контрольної суми UDP. Дізнатися більше про те, чому це відбувається, можна за допомогою пошуку «Розвантаження контрольної суми UDP (UDP checksum offload)».

Крок 4: Очистіть

На цьому кроці, ви вимкнете і очистите Mininet.

- a. У терміналі, який запустив Mininet, введіть **quit** в командному рядку.

```
mininet> quit
```

- b. У рядку введіть **sudo mn -c**, щоб очистити процеси, запущені Mininet.

```
[analyst@secOps ~]$ sudo mn -c
```

Аналіз результатів дослідження

Ця лабораторна робота надала можливість проаналізувати роботу протоколів TCP і UDP із захоплених сеансів FTP і TFTP. В чому полягає різниця керування зв'язком у TCP та UDP?

Основна різниця полягає в надійності та контролі.

- **TCP (Transmission Control Protocol)** — це протокол, орієнтований на з'єднання. Він встановлює надійний сеанс за допомогою трестороннього рукоштовкання, гарантує доставку пакетів у правильному порядку, перевіряє їх цілісність і керує потоком даних (механізм ковзного вікна). Це надійний, але повільніший протокол.
- **UDP (User Datagram Protocol)** — це протокол без встановлення з'єднання. Він просто відправляє пакети (дейтаграми) без гарантії їх доставки, порядку чи цілісності (хоча є необов'язкова контрольна сума). Він дуже швидкий і має мінімальні накладні витрати, що робить його ідеальним для додатків, де швидкість важливіша за надійність (наприклад, DNS, TFTP, онлайн-ігри, стрімінг).

Кінець документа